

2° Instancia Evaluativa

Interfaz Gráfica

Proyecto: Sistema de Autogestión Estudiantil — versión Swing

Información General

- Examen práctico grupal (3-4 integrantes)
- Entrega: 12 junio 2026

¿De qué se trata?

En la primera instancia evaluativa desarrollaron el Sistema de Autogestión Estudiantil como una aplicación de consola en Java. En esta segunda instancia, el objetivo es transformar ese mismo sistema en una aplicación de escritorio con interfaz gráfica, aplicando la arquitectura MVC + DAO y persistiendo los datos en archivos de texto.

Lo que se reutiliza de IE1:

- Estudiante, Materia, InscripcionMateria → pasan al paquete modelo sin cambios estructurales.
- Interfaces Evaluable y Consultable → se mantienen exactamente igual.
- Toda la lógica de negocio (getCondicion, getPromedio, estaAprobada, getMateriasCriticas, etc.).

Lo que se agrega:

- Arquitectura MVC: separación en Modelo, Vista, Controlador y DAO.
- Vista en Swing: interfaz gráfica diseñada en NetBeans con los componentes obligatorios.
- DAO: clases que leen y escriben los datos en archivos .txt.
- Persistencia: los datos se cargan al iniciar y se guardan cuando cambian.

Entregables obligatorios:

CRÍTICO: La falta de cualquiera de los 3 entregables obligatorios anula la instancia evaluativa del alumno

Código del proyecto (obligatorio)

- Link al repositorio de GitHub con el proyecto completo.
- TODOS los integrantes deben tener commits realizados.
 - Penalización: integrante sin commits pierde el 50% de la nota grupal.

Video explicativo (obligatorio)

- Duración: entre 10 y 15 minutos.
- Todos los integrantes con cámara encendida.
- Cada integrante debe mostrar su pantalla con el código al momento de explicar.
- Se debe explicar: la arquitectura MVC+DAO del proyecto, el código técnico de la parte implementada, los conflictos encontrados y cómo los resolvieron, y el uso transparente de IA u otras fuentes.
- Se evaluará especialmente la comprensión individual de cada parte del código implementada.
- Pueden usar Vimeo para capturar pantalla y rostro. Sin webcam: grabarse con el celular y unir con Canva.

Documentación (obligatorio)

- En la carpeta del proyecto debe haber una carpeta con capturas de los prompts utilizados, o un Word con los links a las conversaciones completas con la IA.
- Esta entrega es INDIVIDUAL. Deberán dejar claro de quién es cada captura.
- Captura del diseño de Figma.
- Alternativa: agregar los links de las conversaciones completas y/o diseño en un README.md

README.md (opcional, recomendado)

- Instrucciones de ejecución (versión de NetBeans, JDK, dependencias).
- Descripción de la arquitectura del proyecto y los paquetes.
- Integrantes y rol de cada uno.
- Desafíos encontrados y cómo se resolvieron.

Nota: esta es la parte práctica de la evaluación. El día de la entrega, cada integrante tendrá que realizar un cuestionario individual y entregar el link del repositorio.

Arquitectura obligatoria — MVC + DAO

El proyecto debe estar organizado en las cuatro capas vistas en clase con paquetes separados. No se aceptan proyectos donde la lógica de negocio, la interfaz y el acceso a datos estén mezclados en los mismos archivos.

Paquete / Capa	Responsabilidad
modelo	Las clases de IE1: Estudiante, Materia, InscripcionMateria, Evaluable, Consultable. Pueden adaptarse agregando toTexto() y fromTexto() o mantenerse iguales si se usa serialización.
dao	Una clase por entidad principal que lee y escribe el archivo correspondiente. No conoce la Vista ni el Controlador.
controlador	Recibe las acciones de la Vista, aplica lógica usando el Modelo, llama al DAO para persistir y actualiza la Vista. No contiene código Swing.
vista	JFrame diseñado con el editor de NetBeans. Los listeners solo llaman al Controlador. No modifica datos directamente.

CRÍTICO: Violaciones de arquitectura que anulan el criterio:

- **FileWriter o FileReader dentro de un ActionListener o en la Vista.**
- **new Materia(...) o lógica de validación dentro de la Vista.**
- **El DAO con referencias a componentes Swing (JTable, JTextField, etc.).**
- **El Controlador con imports de javax.swing o llamadas a JOptionPane.**

Persistencia obligatoria — Archivo de flujo

- Un archivo por entidad principal (ej: materias.txt, inscripciones.txt).
- Los datos se cargan al iniciar la aplicación — el DAO lee los archivos en el constructor del Controlador.
- Los datos se guardan cada vez que cambian — el DAO escribe después de cada alta, baja o modificación.
- Si el archivo no existe al iniciar, la app arranca con lista vacía sin lanzar error.
- Los archivos se generan en la carpeta del proyecto, sin rutas absolutas.

Dos formas válidas de persistencia — elegir una:

- **Texto plano** (FileWriter + BufferedReader): agregar toTexto() y fromTexto() a las clases del Modelo.
- **Serialización** (ObjectOutputStream + ObjectInputStream): las clases implementan Serializable y se guarda el ArrayList completo.

Diseño de interfaz — Figma

El diseño de la interfaz es parte del proyecto. Cada grupo debe diseñar en Figma cómo va a verse la aplicación antes de programarla, aplicando los conceptos de UX/UI vistos en clase.

La estética es libre — el grupo elige colores, tipografía y estilo visual. Lo que importa es que las decisiones de diseño estén justificadas por los principios aprendidos.

Entrega:

- El link al archivo de Figma se incluye en el repositorio o en el README junto con el resto del proyecto.
- El diseño no se entrega por separado — forma parte del entregable de código.

Componentes Swing obligatorios

Los siguientes componentes deben estar presentes y funcionalmente integrados — no solo colocados sin uso:

Componente	Uso esperado
JLabel	Etiquetas de campos, títulos de sección, mensajes de estado o feedback (ej: 'Alumno regular', 'Atención: asistencia en riesgo').
JTable	Tabla de materias inscritas con columnas: nombre, condición, asistencia % y promedio. Se actualiza al agregar o dar de baja.
JList	Lista de materias en riesgo (asistencia entre 75% y 85%) o materias aprobadas. Se actualiza junto con la tabla.
JButton	Acciones principales: inscribir materia, dar de baja, registrar asistencia, registrar nota. Al menos tres botones funcionales.
JMenuBar	Menú con al menos: Archivo (cerrar) y Reportes (situación general, materias en riesgo, aprobadas).
JOptionPane	Confirmación antes de dar de baja una materia y al menos un mensaje de alerta cuando la asistencia baja del 75%.
CardLayout	Navegación entre al menos dos secciones: panel principal de materias y panel de reportes o perfil del estudiante.
JScrollPane	Envolviendo la JTable y la JList para que tengan scroll cuando hay muchas materias.

Layouts

- BorderLayout como estructura principal de la ventana (barra lateral o superior + área central).
- Al menos un layout adicional aplicado con criterio dentro de algún panel (GridLayout, FlowLayout, BoxLayout o GridBagLayout).

Funcionalidades mínimas

La aplicación debe mostrar en pantalla la información del sistema de autogestión. No es necesario replicar todas las funcionalidades de consola, pero sí las siguientes:

- Perfil del estudiante — nombre, carrera y año de ingreso visibles en algún panel.
- Inscribir materia — formulario con nombre, código, cuatrimestre y año. Validaciones de IE1 se mantienen (código único, cuatrimestre 1 o 2, código 3-10 chars).
- Dar de baja — eliminar la materia seleccionada con confirmación previa.
- Registrar asistencia — para la materia seleccionada en la tabla, indicar presente o ausente.
- Registrar nota — agregar una nota a la materia seleccionada. Validar rango 0-10 y máximo 5 notas.
- Tabla de materias — muestra todas las materias con condición, asistencia % y promedio. Se actualiza en tiempo real.
- Lista de alertas — JList con las materias cuya asistencia está entre 75% y 85%.
- Reporte accesible desde el menú — al menos el reporte de situación general.
- Estado vacío — mensaje cuando no hay materias inscriptas.

BONUS (puntos extras)

BONUS: Los puntos obtenidos en los ítems extras (al igual que la nota conceptual) se suman al puntaje final del proyecto para mejorar la nota global de la IE

JDBC — Agregar un DAO de base de datos

- Crear una base de datos MySQL con las tablas correspondientes a las entidades del proyecto.
- Crear un DAO exclusivo de la bbdd que use Connection, PreparedStatement y ResultSet.
- El Modelo, el Controlador y la Vista no deben modificarse.
- Incluir en el README las instrucciones para crear la base de datos (script SQL o capturas).

Edición de registros

- Al seleccionar una materia en la tabla, cargar sus datos en un formulario editable.
- Un botón 'Guardar cambios' actualiza el registro con `setValueAt()` y persiste.

Prototipo funcional en Figma

- Convertir el diseño en un prototipo navegable: conectar las pantallas con interacciones reales en Figma.
- El prototipo debe permitir simular el flujo principal sin necesidad de abrir la app de Java.

Funcionalidades adicionales de IE1 migradas a Swing

- Reporte de materias en riesgo con ordenamiento ascendente por asistencia.
- Reporte de materias aprobadas con nota máxima, mínima y promedio del conjunto.
- Búsqueda de materia por código o nombre con resultado resaltado en la tabla.

README.md completo

- Instrucciones claras de ejecución, estructura del proyecto, integrantes con roles y desafíos.

Distribución de puntaje de la 2° IE

El puntaje se compone de dos partes que deben aprobarse de forma independiente:

- Cuestionario individual: 100 puntos → 40% de la nota global (mínimo para aprobar: 55/100)
- Proyecto grupal: 100 puntos → 60% de la nota global (mínimo para aprobar: 55/100)

Si se desaprueba cualquiera de las dos partes, la IE queda desaprobada con un 3 o menos.

Escala de valoración

Nota	Puntaje
1	01 – 24
2	25 – 39
3	40 – 54
4 (aprueba)	55 – 61
5	62 – 66
6	67 – 72
7 (accede a IEFI)	73 – 79
8	80 – 87
9	88 – 96
10	97 – 100